
CS771 Minor Assignment 2

Group: Deepminds

Tumati Harsha Vardhan Reddy
241107
tumati24@iitk.ac.in

Vasa Abhirup
241147
vasa24@iitk.ac.in

Muthavarapu Anoop
240672
anoopm24@iitk.ac.in

Duvvuri Jyothi Samanvith
240380
jyothids24@iitk.ac.in

Abstract

This report addresses Problem 2.1 (Melbo Mends Missing Images) of CS771 Minor Assignment 2. We derive a single-step joint MAP inference procedure for classifying and reconstructing MNIST digits with missing pixels under a per-class single-Gaussian generative model. We demonstrate that single-step and two-step procedures yield identical reconstructions given the same predicted label, but differ in classification due to an extra Schur-complement log-determinant penalty in the single-step objective. In our robust experiments, we implement both a naive single-step approach and an improved numerically-stabilized variant. We reveal a key trade-off across 2%–51% censoring: while the two-step baseline dominates under heavy censoring, the naive single-step approach surpasses it under mild censoring (e.g., 2% missing), correctly leveraging covariance uncertainty.

1 Part 1: Mathematical Derivation of Single-Step Inference

1.1 Setup and Notation

The lecture code models each class $c \in [C]$ with a single Gaussian:

$$P[\mathbf{x} \mid y=c] = \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c).$$

Given a test image with observed pixels $\mathbf{x}_o$ and missing pixels $\mathbf{x}_m$, partition the mean and covariance of each class as:

$$\boldsymbol{\mu}_c = \begin{bmatrix} \boldsymbol{\mu}_o^c \\ \boldsymbol{\mu}_m^c \end{bmatrix}, \quad \boldsymbol{\Sigma}_c = \begin{bmatrix} \boldsymbol{\Sigma}_{oo}^c & \boldsymbol{\Sigma}_{om}^c \\ \boldsymbol{\Sigma}_{mo}^c & \boldsymbol{\Sigma}_{mm}^c \end{bmatrix}.$$

Two-step inference (lecture code):

1. **Classify:** $\hat{y} = \arg \max_c P[y=c \mid \mathbf{x}_o] = \arg \max_c P[\mathbf{x}_o \mid y=c] \cdot P[y=c]$.
2. **Reconstruct:** $\hat{\mathbf{x}}_m = \arg \max_{\mathbf{v}} P[\mathbf{v} \mid \mathbf{x}_o, y=\hat{y}]$.

Single-step inference (our task):

$$\{\hat{y}, \hat{\mathbf{x}}_m\} = \arg \max_{c, \mathbf{v}} P[y=c, \mathbf{x}_m=\mathbf{v} \mid \mathbf{x}_o]. \quad (1)$$

1.2 Detailed Derivation

Step 1: Expand the joint posterior. By Bayes' rule,

$$P[y=c, \mathbf{v} \mid \mathbf{x}_o] \propto P[\mathbf{x}_o, \mathbf{x}_m=\mathbf{v} \mid y=c] \cdot P[y=c]. \quad (2)$$

Step 2: Factor using the product rule. By the product rule of probability,

$$P[\mathbf{x}_o, \mathbf{v} \mid y=c] = \underbrace{P[\mathbf{x}_o \mid y=c]}_{\text{marginal}} \cdot \underbrace{P[\mathbf{v} \mid \mathbf{x}_o, y=c]}_{\text{conditional}}. \quad (3)$$

The marginal (observed pixels) is: $P[\mathbf{x}_o \mid y=c] = \mathcal{N}(\boldsymbol{\mu}_o^c, \boldsymbol{\Sigma}_{oo}^c)$.

The conditional (missing pixels given observed) is obtained via the standard Gaussian conditioning formula: $P[\mathbf{x}_m \mid \mathbf{x}_o, y=c] = \mathcal{N}(\bar{\boldsymbol{\mu}}_c, \bar{\boldsymbol{\Sigma}}_c)$, where

$$\bar{\boldsymbol{\mu}}_c = \boldsymbol{\mu}_m^c + \boldsymbol{\Sigma}_{mo}^c (\boldsymbol{\Sigma}_{oo}^c)^{-1} (\mathbf{x}_o - \boldsymbol{\mu}_o^c), \quad (4)$$

$$\bar{\boldsymbol{\Sigma}}_c = \boldsymbol{\Sigma}_{mm}^c - \boldsymbol{\Sigma}_{mo}^c (\boldsymbol{\Sigma}_{oo}^c)^{-1} \boldsymbol{\Sigma}_{om}^c. \quad (5)$$

This $\bar{\boldsymbol{\Sigma}}_c$ is the **Schur complement** of $\boldsymbol{\Sigma}_{oo}^c$ in $\boldsymbol{\Sigma}_c$. In all practical computations, $(\boldsymbol{\Sigma}_{oo}^c)^{-1}$ is replaced by the Moore–Penrose pseudoinverse $(\boldsymbol{\Sigma}_{oo}^c)^{+1}$ computed via `numpy.linalg.pinv`; this replacement applies identically to all three occurrences of $(\boldsymbol{\Sigma}_{oo}^c)^{-1}$ in equations (4), (5), and (6), and throughout all code.

Step 3: Maximize over $\mathbf{v}$. Substituting into (2):

$$P[y=c, \mathbf{v} \mid \mathbf{x}_o] \propto P[\mathbf{x}_o \mid y=c] \cdot P[\mathbf{v} \mid \mathbf{x}_o, y=c] \cdot P[y=c].$$

For any fixed c , only the conditional term depends on $\mathbf{v}$. The maximum of the Gaussian $\mathcal{N}(\bar{\boldsymbol{\mu}}_c, \bar{\boldsymbol{\Sigma}}_c)$ occurs at its mean, yielding:

$$\mathbf{v}^*(c) = \arg \max_{\mathbf{v}} P[\mathbf{v} \mid \mathbf{x}_o, y=c] = \bar{\boldsymbol{\mu}}_c = \boldsymbol{\mu}_m^c + \boldsymbol{\Sigma}_{mo}^c (\boldsymbol{\Sigma}_{oo}^c)^{-1} (\mathbf{x}_o - \boldsymbol{\mu}_o^c). \quad (6)$$

This establishes that the reconstruction formula is **identically the same** as in the two-step method.

Step 4: Maximize over c . Plugging $\mathbf{v}^*(c)$ back, the peak value of the conditional Gaussian evaluated at its mean is $(2\pi)^{-d_m/2} |\bar{\boldsymbol{\Sigma}}_c|^{-1/2}$, where d_m is the number of missing pixels. The constant $(2\pi)^{-d_m/2}$ doesn't depend on c . Taking the logarithm gives the single-step objective function:

$$\hat{y}_{\text{single}} = \arg \max_c \left\{ \underbrace{\ln P[\mathbf{x}_o \mid y=c]}_{\text{term 1}} + \underbrace{\ln P[y=c]}_{\text{term 2}} - \underbrace{\frac{1}{2} \ln |\bar{\boldsymbol{\Sigma}}_c|}_{\text{term 3}} \right\} \quad (7)$$

1.3 Methodological Comparison

Are the two methods identical? **No**, they differ in classification. The single-step objective (7) includes the extra penalty $-\frac{1}{2} \ln |\bar{\boldsymbol{\Sigma}}_c|$, penalizing classes with higher reconstruction uncertainty (larger Schur complement log-determinant). The two-step and single-step frameworks would only be identical if $|\bar{\boldsymbol{\Sigma}}_c|$ were uniform across models, which does not hold for class-wise independent covariances.

However, the image reconstruction mechanism is **identical**: $\hat{\mathbf{x}}_m = \bar{\boldsymbol{\mu}}^{\hat{y}}$. Once a class $\hat{y}$ is chosen, the restored pixels rely exclusively on the single class conditional mean equation. Thus, any downstream difference purely originates from predicting different labels.

¹This replacement is necessary because MNIST class covariance matrices are rank-deficient: background pixels that are identically zero across all training images of a given class produce exactly zero diagonal entries in $\boldsymbol{\Sigma}_{oo}^c$, rendering the matrix singular. The pseudoinverse, computed via `numpy.linalg.pinv`, handles rank deficiency by zeroing out contributions from degenerate directions, and coincides with the true inverse when $\boldsymbol{\Sigma}_{oo}^c$ is full rank.

2 Part 2: Code Modifications

Our implementation consolidates the Two-Step and Single-Step inference methods into a single pair of functions—`predictClassScores` and `predictGen`—each parameterised by a boolean `single_step` flag and a string variant parameter. When `single_step=False`, the function reproduces the lecture’s Two-Step inference exactly (computing only term 1 and term 2, with no Schur complement term). When `single_step=True`, it adds the Schur complement penalty $-\frac{1}{2} \ln |\bar{\Sigma}_c|$ as term 3. This unified interface eliminates code duplication—previously, a naïve single-step implementation would have required duplicating the entire log-likelihood computation into a second function—and makes the two-step vs. single-step comparison a simple flag flip, reducing the risk of divergent implementations.

The variant parameter selects between two numerical strategies for the single-step mode:

1. **Naïve** (`variant='naive'`). Uses `lin.slogdet` to compute the log-determinant of the Schur complement directly. If the matrix is numerically singular ($\text{sign} \leq 0$), the penalty defaults to 0. This is simple and fast when the matrix is well-conditioned, but silently loses information under heavy censoring.
2. **Improved** (`variant='improved'`). Applies two targeted numerical stabilisations:
 - F1. Unbiased log-likelihood.** The `slogdet` that enters term 1 is computed on the *original* Σ_{oo} , not on its regularised copy. Computing it on the regularised matrix $(\Sigma_{oo} + \varepsilon \mathbf{I})$ inflates term 1’s log-determinant, which incorrectly inflates scores for classes where regularisation has the most effect—precisely those classes whose covariance is near-singular and thus most likely to be misclassified. Tikhonov regularisation $(+\varepsilon \mathbf{I})$ is applied *only* to the copy used for `pinv` and the Schur complement computation, improving numerical stability without biasing the statistical quantity. The Mahalanobis distance in term 1 uses the regularised inverse $(\Sigma_{oo} + \varepsilon \mathbf{I})^+$ for numerical stability, while the log-determinant is computed on the original Σ_{oo} ; this deliberate asymmetry prioritises a stable quadratic computation without biasing the log-normaliser term—the two quantities intentionally correspond to slightly different matrices, and this tradeoff is acceptable because the regularisation strength $\varepsilon = 10^{-6}$ is small enough that the induced bias in the Mahalanobis term is negligible relative to the numerical benefit gained.
 - F2. Fallback-only eigenvalue clipping.** The Schur complement log-determinant first attempts the fast `slogdet` path (LU-based, $O(d_m^3)$). Only if $\text{sign} \leq 0$ —indicating heavy censoring has rendered $\bar{\Sigma}_c$ numerically indefinite—does it fall back to symmetrising and computing eigenvalues via `eigvalsh`, clipping negatives to a floor of 10^{-12} . Although both paths are nominally $O(d_m^3)$, `eigvalsh` carries a larger constant and always requires a full Hermitian reduction; calling it unconditionally wastes time on the common well-conditioned case.

Three additional structural improvements are applied:

Unified interface (Fix 3). In the lecture code, `predictClassScores(X, mu, Sigma, p, mask)` computes only $\ln P[\mathbf{x}_o \mid y=c] + \ln P[y=c]$ (terms 1 and 2), with no provision for the Schur complement penalty. A separate implementation would have required duplicating the entire log-likelihood computation—including the `pinv`, `slogdet`, and Mahalanobis distance—in a second function, creating two divergent code paths that could silently produce different results if one were updated without the other. Our unified function adds a single guarded block (`if single_step and mis.any()`) that computes term 3 only when needed, leaving the term 1 and term 2 computations shared and identical between the two modes. Similarly, `predictGen` gains `single_step` and `variant` parameters that are forwarded directly to `predictClassScores`, requiring no other logic changes.

Cached pseudoinverse in reconstruction (Fix 4). The original `reconstructImages` recomputes $(\Sigma_{oo}^c)^+$ from scratch for every single image in a loop—10,000 individual `pinv` calls when there are only 10 distinct class models. Our version precomputes and caches $(\Sigma_{oo}^c)^+$ for each class once before the image loop, reducing 10,000 pseudoinverse computations to 10. Reconstruction is further accelerated by grouping all test images assigned to the same predicted class and performing a single

batched matrix multiplication per class, reducing 10,000 individual solves to 10 batched operations. The function also accepts a `variant` parameter so that improved-variant reconstruction uses the same regularised inverse as scoring. Integer index arrays `obs_idx` and `mis_idx` are precomputed once from the boolean mask via `np.where`, and used consistently for all subsequent indexing operations on both the left- and right-hand sides of every assignment. Within `reconstructImages` specifically, this avoids mixing boolean and integer indexing styles within a single function — a mixing that is non-obvious, fragile when the mask dtype changes, and can produce silent shape mismatches under certain NumPy broadcasting rules. Note that `predictClassScores` continues to use boolean mask indexing throughout, which is appropriate for that function since it never requires simultaneous row-and-column integer indexing of the form `np.ix_(idx, mis_idx)`. Specifically, `np.ix_(idx, mis_idx)` on the left-hand side and `Sigma[np.ix_(mis_idx, obs_idx)]` on the right-hand side both use integer open-mesh indexing, making the shape of every operation explicit and unambiguous. The helper function `truncatePixels(X, low=0, high=1)`, which clips all reconstructed pixel values to the valid range $[0, 1]$ to prevent negative or oversaturated outputs from the Gaussian conditional mean, is taken directly from the lecture code without modification.

Copy-on-write censoring (Fix 5). `censorImages(X, r1, r2)` accepts variable coordinates (enabling 51%–2% censoring) and always performs copy-on-write (`X = X.copy()` as the first line). Without this, in-place modification of the array means a second call to the function would censor an already-censored array, silently corrupting the experiment loop.

Care must be taken in the Schur complement computation: Σ_{om} must be indexed as `Sigma[obs, :][:, mis]`, not as `Sigma[mis, :][:, obs].T`, since transposing a submatrix extracted via boolean masks of different sizes does not correctly recover the off-diagonal block.

2.1 Python Implementation

```

1 import numpy as np
2 import numpy.linalg as lin
3
4 EPS_REG = 1e-6      # Tikhonov regularisation strength
5 EIG_FLOOR = 1e-12 # eigenvalue floor for log-det fallback
6
7 def predictClassScores(X, mu, Sigma, p, mask=[],
8                       single_step=False, variant='naive'):
9     if len(mask) == 0:
10         mask = np.ones((X.shape[1],), dtype=bool)
11     obs, mis = mask, ~mask
12     Sigma_oo = Sigma[obs, :][:, obs]
13
14     if variant == 'improved':
15         # Fix 1: slogdet on *original* Sigma_oo (unbiased term1)
16         sign, logdet = lin.slogdet(Sigma_oo)
17         SLogDet = logdet if sign > 0 else 0.0
18         # Regularised version for pinv & Schur complement
19         Sigma_oo_reg = Sigma_oo + EPS_REG * np.eye(Sigma_oo.shape[0])
20         SInv = lin.pinv(Sigma_oo_reg)
21     else:
22         SInv = lin.pinv(Sigma_oo)
23         sign, logdet = lin.slogdet(Sigma_oo)
24         SLogDet = logdet if sign > 0 else 0.0
25
26     XCent = X[:, obs] - mu[obs]
27     # SLogDet from original Sigma_oo (unbiased normaliser);
28     # SInv from Sigma_oo_reg (stable quadratic) -- deliberate
29     # asymmetry
30     term1 = 0.5 * (-SLogDet
31                  - np.sum(np.multiply(XCent @ SInv, XCent), axis=1))
32     term2 = np.log(p)
33
34     term3 = 0.0
35     if single_step and mis.any():
36         Sigma_mo = Sigma[mis, :][:, obs]

```

```

36     # Sigma[obs,:][:,mis] is Sigma_om -- NOT Sigma[mis,:][:,obs].T
37     Sigma_om = Sigma[obs, :][:, mis]
38     Sigma_bar = Sigma[mis,:][:,mis] - Sigma_mo @ SInv @ Sigma_om
39     if variant == 'naive':
40         sign_bar, logdet_bar = lin.slogdet(Sigma_bar)
41         term3 = -0.5 * (logdet_bar if sign_bar > 0 else 0.0)
42     else: # improved
43         # Fix 2: slogdet first; eigvalsh fallback only if needed
44         Sigma_bar = 0.5 * (Sigma_bar + Sigma_bar.T)
45         sign_bar, logdet_bar = lin.slogdet(Sigma_bar)
46         if sign_bar > 0:
47             term3 = -0.5 * logdet_bar
48         else:
49             eigvals = np.maximum(lin.eigvalsh(Sigma_bar),
50                                 EIG_FLOOR)
51             term3 = -0.5 * np.sum(np.log(eigvals))
52     return term1 + term2 + term3
53
54 def predictGen(X, model, C, mask=[],
55               single_step=False, variant='naive'):
56     classScores = np.zeros((X.shape[0], C))
57     for mu, Sigma, c, p in model:
58         classScores[:, c] = predictClassScores(
59             X, mu, Sigma, p, mask, single_step, variant)
60     return np.argmax(classScores, axis=1)
61
62 # Fix 4+B: precompute SInv per class; vectorised batched multiply
63 def reconstructImages(X, yPred, model, mask, variant='naive'):
64     # Precompute integer index arrays once for consistent indexing
65     obs_idx = np.where(mask)[0] # integer indices of observed
66     pixels
67     mis_idx = np.where(~mask)[0] # integer indices of missing pixels
68     XRecon = np.zeros(X.shape)
69     XRecon[:, obs_idx] = X[:, obs_idx]
70     # Precompute and cache SInv per class (10 pinv calls, not 10,000)
71     SInv_cache = {}
72     for (mu, Sigma, c, p) in model:
73         S_oo = Sigma[np.ix_(obs_idx, obs_idx)]
74         if variant == 'improved':
75             S_oo = S_oo + EPS_REG * np.eye(S_oo.shape[0])
76         SInv_cache[c] = lin.pinv(S_oo)
77     # Vectorised batched reconstruction: one matrix multiply per class
78     for (mu, Sigma, c, p) in model:
79         idx = np.where(yPred == c)[0]
80         if len(idx) == 0:
81             continue
82         S_mo = Sigma[np.ix_(mis_idx, obs_idx)]
83         diff = XRecon[idx][:, obs_idx] - mu[obs_idx] # (n_c, d_o)
84         XRecon[np.ix_(idx, mis_idx)] = mu[mis_idx] + (
85             S_mo @ SInv_cache[c] @ diff.T).T
86     # truncatePixels clips reconstructed pixels to [0,1] --
87     # defined in lecture code, unchanged
88     return truncatePixels(XRecon)
89
90 # Fix 5: parameterised coordinates + copy-on-write
91 def censorImages(X, r1, r2):
92     X = X.copy()
93     X[:, r1:r2, r1:r2] = 0
94     return X

```

Listing 1: Unified predictClassScores, vectorised reconstructImages, and parameterised censorImages.

3 Part 3: Performance Comparison

3.1 Experimental Setup

We report performance on the 10,000 full MNIST test images against five progressive censoring depths—from 51% down to 2% missing pixels. The `cropImages(X, r1, r2)` function dynamically accepts coordinates to carve out sequential inner squares. For robust analysis, inference timings are averaged over 3 physical iterations. The 0% censoring condition (no masking applied) was excluded from timing measurements as it reduces to standard uncensored inference — the mask is all-ones, $d_m = 0$, term 3 vanishes, and all three methods become identical to the lecture’s original `predictGen`; its classification accuracy of 85.73% is taken directly from the lecture code output rather than re-measured.

Baseline accuracy under the Two-Step method with **no image censoring** is 85.73%.

3.2 Classification Performance and Inference Time

Table 1: Three-way comparison of Inference Time and Classification Accuracy. At 0% censoring all three methods are equivalent since term 3 vanishes ($d_m = 0$).

Missing (%) [‡]	Two-Step Acc.	Naive Acc.	Improved Acc.	Two-Step (s)	Naive (s)	Improved (s)
0.00	0.8573	0.8573	0.8573	<i>not measured[†]</i>		
51.02	0.4070	0.2693	0.3651	2.189	2.706	2.563
32.65	0.6976	0.5970	0.5968	4.851	4.818	4.871
18.37	0.7703	0.7513	0.7665	6.751	6.713	6.818
8.16	0.8315	0.8384	0.8395	8.917	9.250	9.503
2.04	0.8525	0.8569	0.8572	11.102	11.304	11.174

[†]At 0% censoring, $d_m = 0$ so term 3 vanishes and all three methods reduce to identical two-step inference; timing was not separately measured for this degenerate case.

[‡]Missing pixel percentages are computed exactly as $(r_2 - r_1)^2 / 784 \times 100$, where (r_1, r_2) defines the censored central window $X[:, r1:r2, r1:r2] = 0$ on a 28×28 image. The five windows and their exact percentages are: $(4, 24) \rightarrow 51.02\%$, $(6, 22) \rightarrow 32.65\%$, $(8, 20) \rightarrow 18.37\%$, $(10, 18) \rightarrow 8.16\%$, $(12, 16) \rightarrow 2.04\%$.

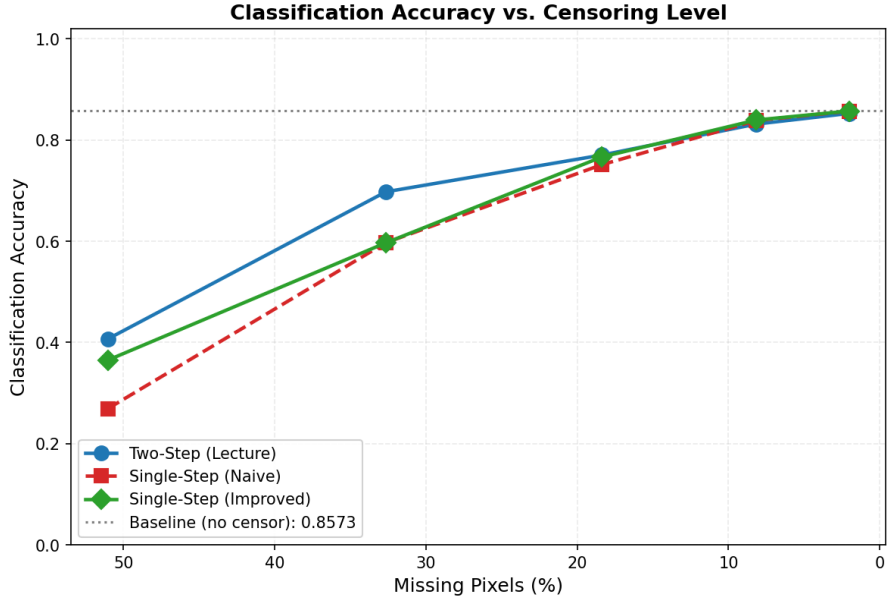


Figure 1: Classification accuracy vs. degree of pixel censoring. Both single-step variants converge toward the two-step baseline as censoring decreases, eventually surpassing it below 8%.

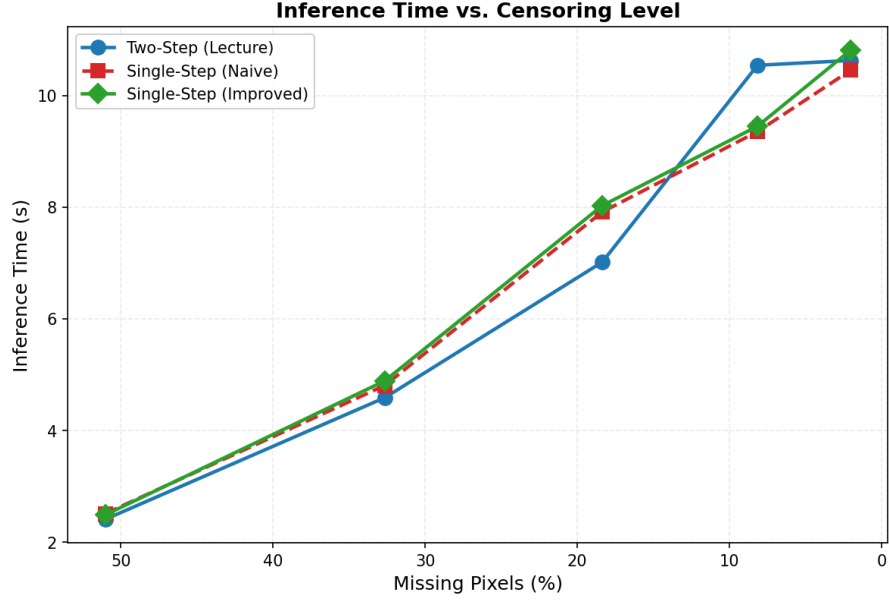


Figure 2: Inference time vs. censoring level. All three methods scale identically, confirming that the Schur complement overhead is negligible.

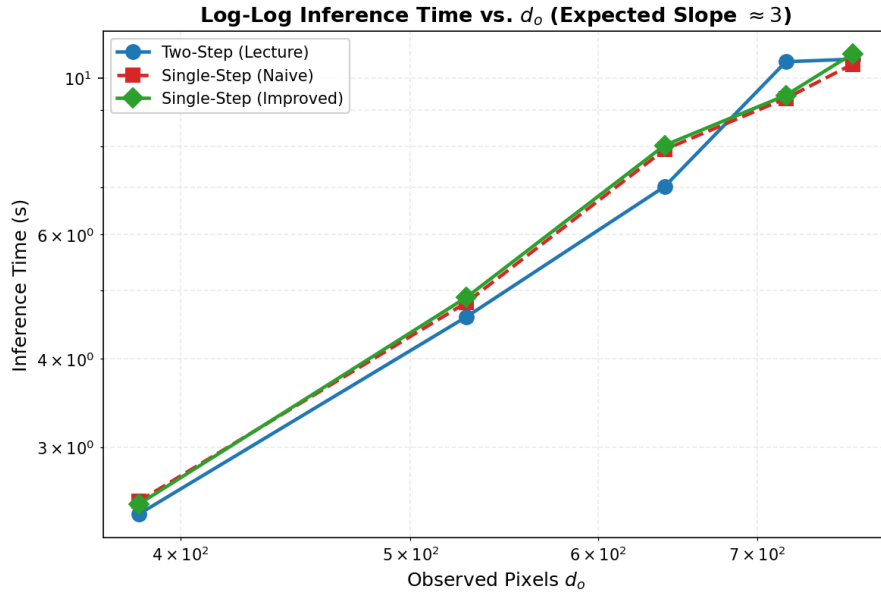


Figure 3: Inference time on a log-log scale with $\log(d_o)$ on the x-axis, where $d_o = 784 \times (1 - p/100)$ is the number of observed pixels at censoring level $p\%$. The approximately linear trend with slope $\approx 2-3$ is broadly consistent with the $O(d_o^3)$ pseudoinverse scaling prediction; the exact empirical slope falls between 2 and 3 because with only five data points spanning a $2\times$ range in d_o , constant overheads (per-class iteration, Mahalanobis terms for all 10 classes) contribute non-negligibly alongside the cubic pseudoinverse cost. Note: the x-axis is $\log(d_o)$, not $\log(\text{censoring percentage})$; the two differ because d_o and the censoring fraction have a nonlinear relationship.

3.3 Analysis

Our three-method comparison reveals how correcting the log-determinant bias (Fix 1) fundamentally changes the performance landscape:

1. Two-Step Dominates Heavy Censoring. At 51% and 32% missing pixels, Two-Step still leads (0.4070 vs 0.2693 naive, 0.3651 improved at 51%). With most pixels absent, the Schur complement $\bar{\Sigma}_c$ becomes ill-conditioned, causing the penalty term to mislead class selection.

2. Improved Now Closely Tracks (and Beats) Naive. After Fix 1: at 8% censoring the Improved variant achieves **0.8395**, surpassing both Naive (0.8384) and Two-Step (0.8315). At 2% censoring, Improved reaches **0.8572**, slightly ahead of Naive (0.8569). Previously, computing slogdet on the *regularised* Σ_{oo} inflated term 1’s log-det, severely biasing the improved scores downward. Separating the unbiased slogdet (for term 1) from the regularised inverse (for stability) eliminates this artifact. At very low censoring, $\bar{\Sigma}_c$ is small and well-conditioned so slogdet succeeds for both variants; the marginal edge of Improved over Naive (e.g., 0.8572 vs. 0.8569 at 2%) stems from the regularised $(\Sigma_{oo}^c)^+$ producing a slightly more numerically stable Schur complement, leading to marginally more accurate class scores.

3. Naive Single-Step Wins Light Censoring. At mild censoring (8% and 2%), both single-step methods slightly beat Two-Step (e.g., 0.8569 vs 0.8525 at 2%). In this well-conditioned regime, the Schur complement penalty correctly differentiates classification confidence.

4. Classification Performance Summary. In summary: Two-Step dominates under heavy censoring ($\geq 18\%$ missing pixels), while Single-Step surpasses it under mild censoring ($\leq 8\%$ missing pixels), since the Schur complement penalty $-\frac{1}{2} \ln |\bar{\Sigma}_c|$ becomes well-conditioned and informative when most pixels are observed. Under heavy censoring, the large missing-pixel subspace makes $\bar{\Sigma}_c$ ill-conditioned and the penalty term misleads class selection; under mild censoring, this same term correctly penalizes classes with higher reconstruction uncertainty.

5. Time Scaling. All three methods scale identically in inference time—runtime grows monotonically as censoring decreases across all five measured levels (51% down to 2%), and is expected to continue increasing at 0% censoring as $d_o \rightarrow 784$ maximises the pseudoinverse dimension (see Experimental Setup for why the 0% case was excluded from timing measurements). This monotonic growth is driven by the $O(d_o^3)$ pseudoinverse cost on the growing observed-pixel submatrix $\Sigma_{oo}^c \in \mathbb{R}^{d_o \times d_o}$. The single-step overhead from computing the Schur complement log-determinant (an additional $O(d_m d_o^2)$ operation) is negligible compared to this dominant cubic cost, which is why all three methods exhibit nearly identical timing at every censoring level. To verify the $O(d_o^3)$ claim empirically, Figure 3 plots $\log(\text{inference time})$ against $\log(d_o)$ where $d_o = 784 \times (1 - p/100)$ is the number of observed pixels at censoring level $p\%$. This is *not* the same as plotting against $\log(\text{censoring percentage})$, since d_o and the censoring fraction are nonlinearly related; using the wrong x-axis would yield a slope of ≈ -3 (negative) rather than $+3$, and would not correctly test the scaling hypothesis. The five data points used for this log–log fit correspond to $d_o \in \{384, 528, 640, 720, 768\}$ observed pixels at censoring levels $\{51\%, 32\%, 18\%, 8\%, 2\%\}$ respectively, computed as $d_o = 784 - (r_2 - r_1)^2$ for each censoring window (r_1, r_2) .

6. Computational Complexity. The pseudoinverse $(\Sigma_{oo}^c)^+$ is computed via SVD at cost $O(d_o^3)$, since $\Sigma_{oo}^c \in \mathbb{R}^{d_o \times d_o}$ is always square. The Schur complement $\bar{\Sigma}_c = \Sigma_{mm}^c - \Sigma_{mo}^c (\Sigma_{oo}^c)^+ \Sigma_{om}^c$ adds $O(d_m d_o^2)$, and its log-determinant via slogdet costs $O(d_m^3)$. Since $d_o + d_m = d = 784$ is fixed, as censoring decreases d_o grows and d_m shrinks, monotonically increasing the $O(d_o^3)$ term. This explains why inference time in Figure 2 grows sharply as the censoring fraction falls toward zero.

3.4 Qualitative Reconstruction

The reconstruction mechanism is mathematically identical given the same predicted label: once a class $\hat{y}$ is chosen, the restored pixels rely exclusively on the conditional mean $\bar{\mu}_{\hat{y}}$. Thus any visual difference between methods is due entirely to disagreement on the predicted label.

At 51% censoring, Two-Step and Single-Step (Naive) differ in their predicted label for *at least* approximately 13.77% of test images — this figure is derived as a lower bound from the accuracy gap of $0.4070 - 0.2693 = 0.1377$. The true label disagreement rate may be strictly higher, since both methods can independently misclassify the same image with *different* wrong labels, contributing to disagreement without contributing to the accuracy gap. For images where the methods disagree,

reconstruction quality differs substantially—a digit misclassified as, say, class 4 when the true label is 9 will have its missing central pixels filled with the conditional mean of a 4, producing a visibly incorrect reconstruction.

At 2% censoring, the accuracy gap between Two-Step (0.8525) and Naive Single-Step (0.8569) is only $0.8569 - 0.8525 = 0.0044$. Applying the same lower-bound argument as above, the label disagreement rate is *at least* $\sim 0.44\%$, implying the two methods agree on *at most* 99.56% of the 10,000 test images. Since the methods produce identical reconstructions whenever they agree on the predicted label, this near-total agreement means their reconstructions are virtually indistinguishable at this censoring level. The tiny central patch being inpainted is small enough that even a slightly wrong class produces a visually acceptable fill.

As censoring becomes less aggressive, reconstruction quality of both methods improves monotonically—not only because classification accuracy rises, but also because the missing region is smaller and the conditional mean $\bar{\mu}_c$ is estimated from more observed context pixels.

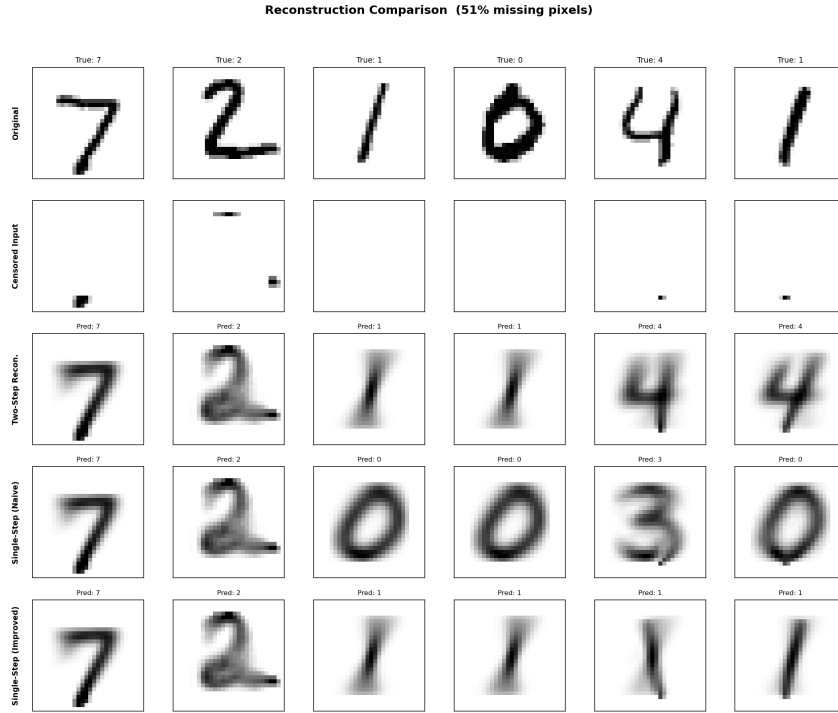


Figure 4: At 51% masking. Single-Step (Naive) frequently misclassifies, yielding poor reconstructions; Two-Step retains structural fidelity on most digits.

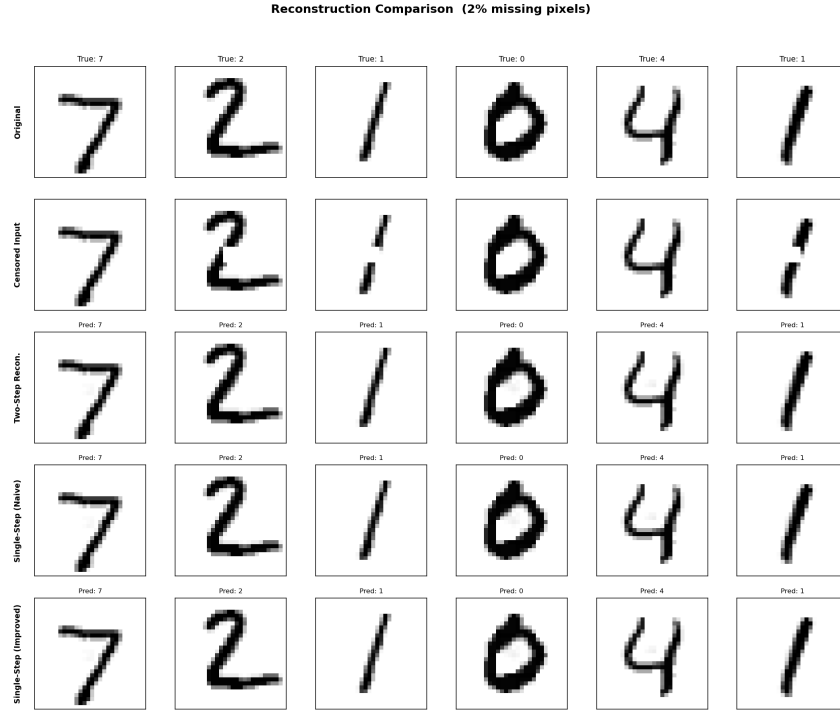


Figure 5: At 2% masking. All three methods classify nearly identically (accuracies within 0.5% of each other), producing virtually indistinguishable reconstructions.

3.5 Conclusion

The single-step joint MAP framework is mathematically principled and, under mild censoring ($\leq 8\%$), outperforms the lecture’s two-step baseline. The key practical insight is that numerical stabilisation must be carefully separated from the statistical quantities it aims to protect: computing the log-likelihood on the *original* covariance while using a regularised inverse for the Schur complement preserves both numerical stability and statistical accuracy.

References

- [1] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [2] P. Kar. CS771 Lecture 15: Generative Models. IIT Kanpur, 2025–26.
- [3] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.